



Carrera **Desarrollo y Diseño Web**

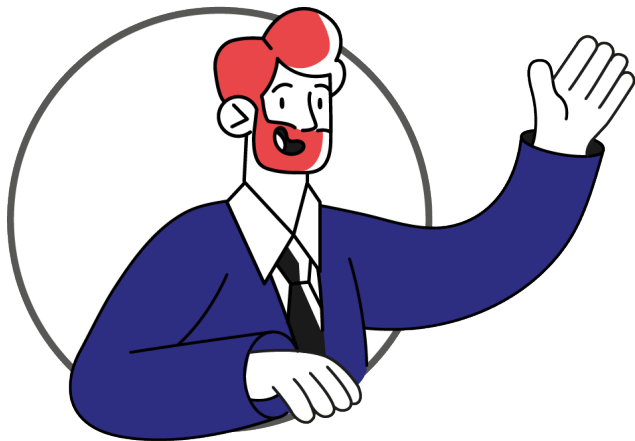
Experiencia 2 – Semana 3

**Interactuando con JavaScript y los
navegadores**

Índice

Introducción a la semana.....	3
Resultado de aprendizaje.....	4
Conceptos relevantes.....	5
Preguntas activadoras.....	5
Actividad.....	6
Objetos del BOM.....	8
WINDOW.....	8
SCREEN.....	31
NAVIGATOR.....	32
Cierre de la semana.....	38
Referencias.....	39
Apuntes.....	40

Introducción a la semana



Como ya has podido comprobar, la programación en JavaScript te permite crear aplicaciones web interactivas y dinámicas. Hasta ahora, has aprendido las bases de la programación, como el uso de variables, arreglos, bucles y operadores de decisión, lo que te ha proporcionado una sólida base para avanzar.

Ahora es el momento de explorar el mundo del Browser Object Model (BOM), que te permitirá interactuar con el navegador y realizar tareas complejas.

Dentro del BOM, conocerás conceptos clave como 'window', que representa la ventana del navegador y te proporciona acceso a numerosas funciones y propiedades. También aprenderás sobre 'screen', que te brinda información sobre la pantalla del usuario, 'history', que te permite manipular el historial de navegación, y 'print', que te permite imprimir contenido desde JavaScript. Además, explorarás la interacción con el usuario a través de ventanas emergentes utilizando 'confirm'.

Asimismo, profundizarás en la gestión del tiempo en JavaScript con las funciones 'setTimeout' y 'setInterval', que te permiten ejecutar código después de un retraso o en intervalos regulares. Y no te preocupes, también aprenderás a controlar estos temporizadores con 'clearTimeout' y 'clearInterval'.

Con estos conocimientos, podrás crear aplicaciones web más interactivas y atractivas.

¡Vamos a explorar el BOM y llevar tus habilidades de programación a un nuevo nivel!

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Utiliza elementos básicos de programación en JavaScript en sitio web, de acuerdo criterios de usabilidad y accesibilidad y los requerimientos del proyecto.

RA2. Programa en JavaScript, considerando el control de elementos BOM (browser) y DOM (documento) para un sitio web bajo criterios de usabilidad, accesibilidad y estéticos.

Indicadores de logro:

IL1. Utiliza correctamente los elementos básicos de programación como variables, arrays, operadores (if - else) y bucles, a fin de dar solución a los requerimientos del proyecto.

IL2. Utiliza el lenguaje JavaScript para programar funciones que permiten reutilizar código de manera eficiente.

IL3. Distingue las funciones y aplicaciones del BOM y DOM en el proceso de desarrollo de un sitio web con JavaScript.

IL4. Reconocer propiedades del BOM y manipular elementos del DOM, a fin de mejorar la usabilidad, accesibilidad y estética del sitio web.

Conceptos relevantes

	BOM	Navegador	Cliente
	Window		

Preguntas activadoras

- ¿Te gustaría saber cómo obtener información sobre la pantalla y el dispositivo del usuario para personalizar la experiencia en tu sitio web?
- ¿Quieres descubrir cómo manipular un historial de navegación para crear una navegación fluida en tu aplicación web?
- ¿Te interesaría aprender a imprimir contenido directamente desde JavaScript y proporcionar una funcionalidad de impresión personalizada en tu sitio web?
- ¿Estás interesado en crear ventanas emergentes y cuadros de diálogo para interactuar con el usuario de manera efectiva?

Actividad

Descripción de la actividad

En esta actividad formativa tendrás que crear una página web utilizando HTML y CSS que contenga un currículum, y luego modificarla para una versión imprimible utilizando JavaScript. Esto incluye la creación de una hoja de estilo específica para impresión y una función JavaScript que gestiona la impresión de la página.

¿Qué es el BOM?

El Browser Object Model (BOM) es un conjunto de objetos y propiedades que proporciona acceso a la funcionalidad y la información relacionada con el navegador web y su ventana. Es fundamental para crear aplicaciones web interactivas y para mejorar la experiencia del usuario.

A través del BOM, se pueden realizar acciones como:

1. Manipular la ventana del navegador, incluyendo abrir, cerrar y redirigir ventanas o pestañas.
2. Controlar y modificar la ubicación actual de la página web, lo que permite la redirección y el seguimiento de enlaces.
3. Interactuar con el historial de navegación del usuario, permitiendo la navegación hacia atrás y adelante en el historial.
4. Obtener información sobre la pantalla del usuario, como resolución y profundidad de color.
5. Generar impresiones en la página actual o en ventanas emergentes.
6. Obtener información sobre el navegador del usuario, como su nombre y versión.
7. Mostrar cuadros de diálogo al usuario para confirmar acciones.

Se utiliza para crear ventanas emergentes, implementar sistemas de confirmación, gestionar cookies, redirigir páginas, controlar temporizadores y realizar otras acciones que involucran la interacción del usuario con el navegador.

Conozcamos ahora algunos de sus objetos más importantes.

Objetos del BOM

Los objetos del BOM son componentes que permiten gestionar elementos diversos en el navegador. Los principales tres objetos del BOM son window, screen, y navigator, y cada uno tiene su propio conjunto de propiedades y métodos para realizar diferentes tareas. A continuación, revisaremos cada uno de ellos.

WINDOW

El objeto window representa la **ventana del navegador y proporciona acceso a muchas funciones y propiedades** relacionadas con la ventana del navegador, como el tamaño de la ventana, la ubicación actual del usuario, métodos para abrir y cerrar ventanas emergentes, y más. Dado su tamaño, lo separaremos en las siguientes unidades:

- **Mensajes:** incluye métodos como 'alert', 'confirm' y 'prompt', que se utilizan para abrir ventanas de alerta, confirmación y para solicitar información al usuario, respectivamente.
- **Impresión:** funciones relacionadas con la impresión de contenido desde la ventana del navegador.
- **Ventanas:** control y manipulación de ventanas y pestañas del navegador.
- **Ubicación:** gestión de la URL y la ubicación del usuario en el navegador.
- **Historial:** funciones para interactuar con el historial de navegación del usuario.
- **Manejo del tiempo:** métodos para controlar la ejecución del código en el tiempo.

Revisemos en profundidad cada uno de ellos y sus ejemplos correspondientes:

WINDOW – Mensajes

A continuación, se describen los métodos del objeto WINDOW relacionados con mensajes:

Tabla 1

Métodos del objeto WINDOW – Mensajes

Método	Descripción	Sintaxis
alert	Abre ventanas de alerta.	alert(mensaje)
confirm	Abre ventanas de confirmación.	let resp =confirm(mensaje)
prompt	Abre ventanas que permiten el ingreso de datos.	let input =prompt(mensaje)

Si deseas agregar un salto de línea al texto que despliegan las ventanas generadas por los métodos del objeto WINDOW, como 'alert', 'confirm' y 'prompt', debes usar \n.

En el ejemplo de la figura 1, se utiliza el método 'confirm'. Para desplegar un mensaje de confirmación que contenga un salto de línea, deberías escribir el mensaje con \n en el punto donde deseas que ocurra el salto.

Figura 1

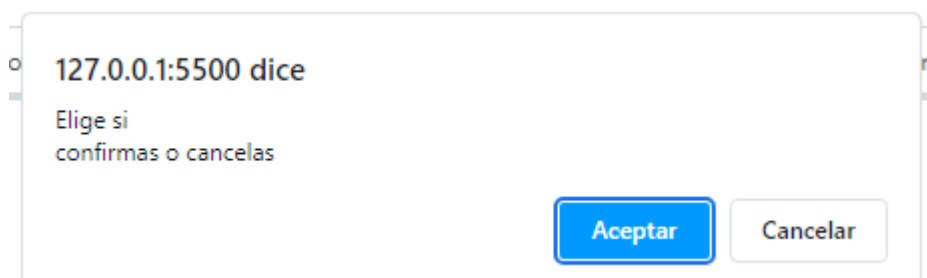
Ejemplo de CONFIRM

```
let respuesta = confirm("Elige si\nconfirmas o cancelas");
```

Lo anterior, se verá tal como se muestra en la figura 2: el mensaje se mostrará en dos líneas distintas en la ventana de diálogo.

Figura 2

Ejemplo de despliegue de CONFIRM



Veamos ahora algunos ejemplos relacionados con los diversos métodos asociados a los mensajes de Windows.

Ejemplos relacionados con el uso del método alert:

La figura 3 muestra la definición de una función llamada alerta. Esta función, cuando se invoca, ejecuta el método alert de JavaScript que a su vez muestra una ventana de diálogo con el mensaje "Hola. Esta es una alerta".

Figura 3

Ejemplo de alerta

```
function alerta() {  
    alert("Hola. Esta es una alerta");  
}
```

En la figura 4 se muestra cómo se vería el resultado en la interfaz del navegador después de que se ha ejecutado la función. Es decir, una ventana emergente de alerta mostrando el mensaje especificado.

Figura 4

Ejemplo de despliegue de alerta



Para ejecutar esta función, necesitamos una acción que la gatille. En este caso, la acción que gatilla la función `alerta()` es un clic del usuario. Esto lo podemos conseguir de las siguientes maneras, tal como se muestra en el ejemplo de la figura 5:

1. **Usando la propiedad href del link:** en este caso, debemos comenzar con "javascript:" y luego, colocamos el nombre de la función que vamos a llamar. De este modo, al hacer clic en un enlace, se ejecuta la función JavaScript especificada.
2. **Usando la propiedad onclick en el link:** consiste en asignar una función JavaScript a la propiedad "onclick" de un enlace. Así, al presionar el link, se gatilla la función que está definida en la propiedad.
3. Existe una tercera forma, que consiste en **monitorear eventos**, pero la veremos más adelante.

Figura 5

Ejemplo de cómo gatillar acciones

```
<p>
  <a href="javascript:alerta();">alerta dentro de href</a>
</p>
<p>
  <a href="#" onclick="alerta();">alerta con onclick</a>
</p>
```

Ejemplos relacionados con el uso del método confirm

En el ejemplo de la figura 6, el código crea la función **confirm**. La primera línea establece una variable llamada **respuesta**, y le asigna como valor la respuesta que dé el usuario. También define que la ventana desplegará el texto “Elige si confirmas o cancelas” y el código es capaz de revisar el valor de la respuesta. En el caso de confirmación, siempre es verdadero o falso.

El operador condicional pregunta **if (respuesta)**, lo que equivale a decir “si la variable es verdadera”. Si es así, envía una alerta con el texto de confirmación. En caso contrario, avisa que fue cancelado.

Figura 6

Ejemplo código de la función CONFIRM

```
function confirma() {  
    let respuesta = confirm("Elige si confirmas o cancelas");  
    if (respuesta) {  
        alert("Has confirmado");  
    } else {  
        alert("Has cancelado");  
    }  
}
```

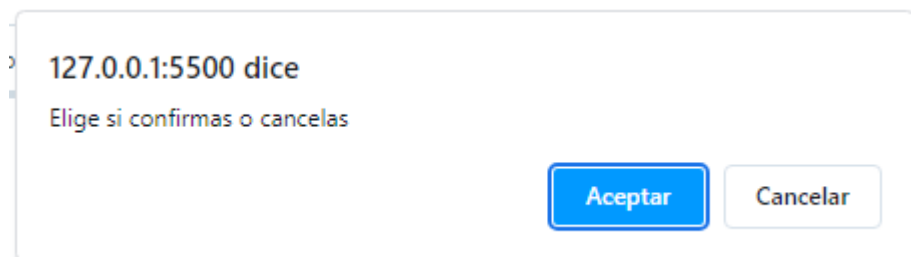
Considerando el código de la figura 6, donde se define la lógica que se activa con la confirmación del usuario, revisemos los posibles despliegues según la elección del usuario.

Despliegue de la función confirm

Esta función es utilizada para obtener una confirmación del usuario. En la figura 7 se muestra cómo se ve el despliegue de la función confirm. En este caso, se muestra el mensaje “Elige si confirmas o cancelas” y las opciones “Aceptar” y “Cancelar” como posibles respuestas.

Figura 7

Ejemplo de despliegue de CONFIRM



Despliegue al elegir opción aceptar

Dependiendo de la respuesta del usuario, se pueden desencadenar diferentes acciones. Al elegir la opción aceptar, la función devuelve un texto de confirmación en la interfaz del usuario, tal como se muestra en la figura 8. Esto significa que la función `confirm()` procedió ejecutó el bloque de código dentro de la condición `if (respuesta)`, que a su vez desencadenó la ventana de alerta con el mensaje "Has confirmado".

Figura 8

Ejemplo de respuesta al confirmar

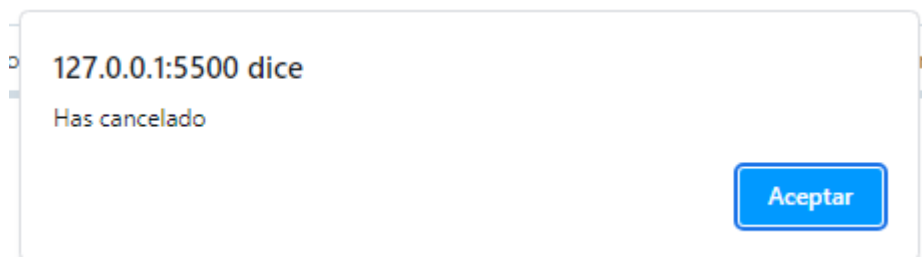


Despliegue el elegir opción cancelar

Siguiendo el flujo del código JavaScript de la función `confirm()`, al elegir la opción cancelar, la ejecución del código pasa al bloque `else` y ejecuta la función `alert("Has cancelado")`, que es la ventana de alerta que vemos en la figura 9.

Figura 9

Ejemplo de respuesta al cancelar



Ejemplos relacionados con el uso del método prompt:

En la figura 10 se muestra una función JavaScript llamada `ingresa()`, que usa el método `prompt` para solicitar al usuario que ingrese su nombre con el mensaje "Por favor, ingresa tu nombre:".

Figura 10

Ejemplo de PROMPT

```
function ingresa() {  
    let respuesta = prompt("Por favor, ingresa tu nombre:");  
    if (respuesta !== null) {  
        if (respuesta.trim() !== "") {  
            alert("Hola, " + respuesta + ". ¡Bienvenido!");  
        } else {  
            alert("No ingresaste tu nombre. ¡Adiós!");  
        }  
    } else {  
        alert("No ingresaste tu nombre. ¡Adiós!");  
    }  
}
```

Como se observa en el código de la figura 10, la función crea la variable **respuesta** y a esta, se le asigna el valor que el usuario ingrese. Luego, se revisa que el usuario haya ingresado algo, con la siguiente condición: **nombre !== null**. Esto quiere decir: "si nombre es distinto a null", donde NULL significa NULO. Una variable declarada, pero que no tiene un valor asignado es considerada nula.

Ahora bien, **nulo no es igual a vacío**. Por ello, en caso de que la variable no sea nula, hacemos una segunda comprobación, si está vacía o no: **if (respuesta.trim() !== "")**



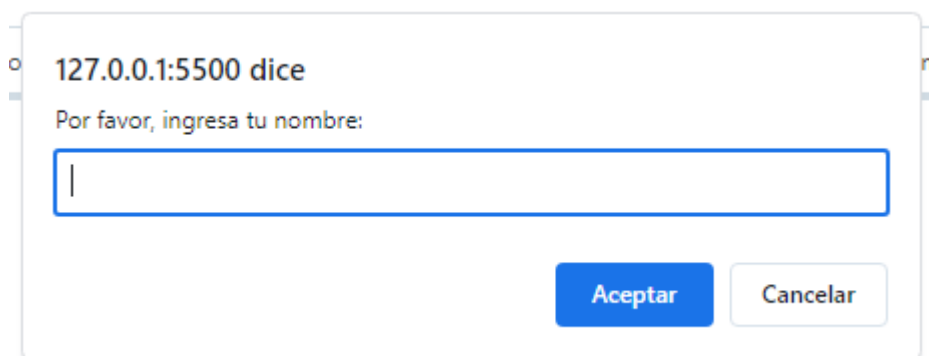
Nota que al final de la variable se agrega **trim**. Esto hace que se quiten los espacios al inicio y al final del texto que contiene la variable. Sólo tras esto se hace la comparación: que no esté vacío (para eso se usan las comillas dobles: "")

A continuación, revisaremos los posibles resultados al ejecutar la función, dependiendo de la interacción del usuario con la ventana de diálogo generada por prompt():

La figura 11 muestra la ventana de diálogo del prompt que aparece cuando se llama a la función `ingresa()`, solicitando al usuario que ingrese su nombre.

Figura 11

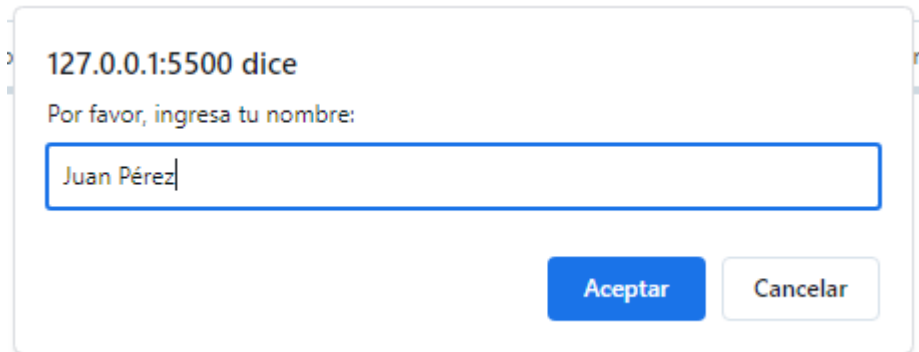
Ejemplo de despliegue de PROMPT



La figura 12 muestra cómo se vería la ventana de diálogo al agregar el texto. En este caso, se ha ingresado: Juan Pérez.

Figura 12

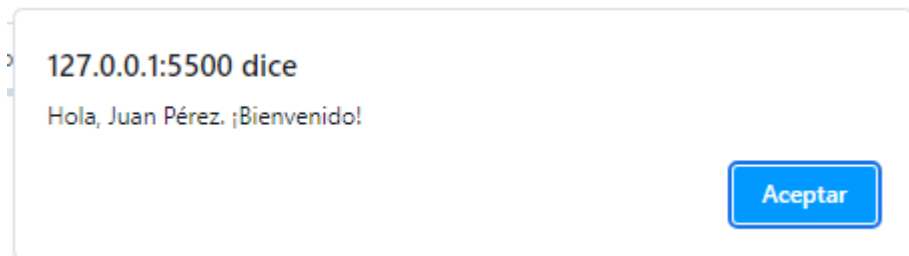
Ejemplo de ingreso de información en PROMPT



Cuando el usuario introduce su nombre y presiona “aceptar” en el paso anterior, se ejecuta el primer bloque de código if dentro de la función, que muestra un mensaje de bienvenida. En este caso, y como vemos en la figura 13, se despliega una alerta con el mensaje: Hola, Juan Pérez. ¡Bienvenido!”.

Figura 13

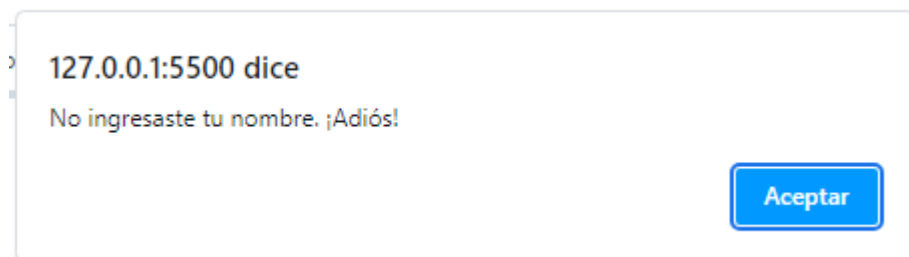
Ejemplo de respuesta de PROMPT según lo programado.



Si se presiona “aceptar” sin agregar texto en la caja del prompt, el resultado es tal como se muestra en la figura 14. Es decir, una alerta con el mensaje: "No ingresaste tu nombre. ¡Adiós!”

Figura 14

Ejemplo de respuesta de PROMPT al no ingresar información.



WINDOW - Impresión

El método **print()** se utiliza para imprimir el contenido de una página web directamente desde el navegador. Este método se invoca en el objeto window del navegador y permite que los usuarios impriman el contenido visible en la página web en ese momento.

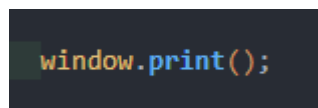


El método **print()** es especialmente útil cuando deseas proporcionar a los usuarios la capacidad de imprimir el contenido de tu página web, como artículos, formularios o informes. Puedes usarlo junto con estilos CSS para controlar la apariencia de la página impresa y garantizar que el resultado impreso se vea de la manera deseada.

La **sintaxis para utilizar este método es window.print()**. Cuando se llama a **window.print()**, el navegador abrirá la ventana de diálogo de impresión estándar que permite a los usuarios seleccionar una impresora y ajustar las opciones de impresión según sus preferencias. Luego, se imprimirá el contenido de la página actual en el papel.

Figura 15

Sintaxis función window.print()



La figura 16 muestra un código HTML de una página web con la funcionalidad de impresión. El botón (<button>) tienen un atributo onclick que, cuando se hace clic, invoca la función window.print(). Es decir, abre la ventana de diálogo de impresión estándar del navegador, permitiendo al usuario imprimir la página web actual.

Figura 16

Ejemplo de código HTML de una página que permite imprimir contenido

```
<!DOCTYPE html>
<html>

<head>
  <title>Ejemplo de Impresión</title>
</head>

<body>
  <h1>Mi Página para Imprimir</h1>
  <p>Este es el contenido de la página que puedes imprimir.</p>

  <button onclick="window.print()">Imprimir</button>
</body>

</html>
```

WINDOW - Ventanas

Dentro del objeto window existen varios métodos que se utilizan para la gestión de ventanas y la manipulación de la interfaz de usuario en una aplicación web. Estos métodos te permiten controlar ventanas emergentes, cambiar la ubicación de la ventana, abrir nuevas ventanas, entre otros.

Tabla 2

Métodos para manipular ventanas

Método	Descripción	Sintaxis
open	Permite abrir nueva ventana.	Let ventana = window.open("url","nombre", atributos)
close	Se utiliza para cerrar ventanas. Sólo funciona si la ventana fue abierta con javascript y no en una pestaña o ventana principal del navegador.	ventana.close()
opener	Indica si ventana está abierta.	let bool = ventana.opener
closed	Indica si ventana está cerrada	Let bool = ventana.closed

Los métodos **close**, **opener** y **closed** funcionan de forma directa. Es decir, no requieren información adicional. Sin embargo, el método **open** requiere 3 parámetros:

- **URL:** la dirección de la página que se quiere abrir.
- **Nombre:** corresponde al nombre que aparecerá como título de la página.
- **Atributos:** opciones que permiten activar o desactivar elementos de la ventana, de modo que sea mostrados u ocultos.

Los atributos especifican cómo se debe comportar la nueva ventana, incluyendo su tamaño, si ciertas barras y menús deben mostrarse, o no, si es redimensionable o si debe tener barras de desplazamiento.

Revisemos cuáles son los atributos de **open**:

Tabla 3

Atributos de open

Atributo	Elemento
Toolbar = [yes no]	Barra de herramientas
Location = [yes no]	Barra de direcciones
Directories = [yes no]	Historial
channelmode = [yes no]	Barra de canales
menubar = [yes no]	Barrade menús
scrollbars = [yes no]	Barra de scroll
resizable = [yes no]	Define si se permite que el usuario cambie el tamaño de la ventana
Width = pixels	Ancho, en pixels
Height = pixels	Alto, en pixels
fullscreen = [yes no]	Abrir ventana maximizada
Top = pixels	Posición desde lado superior, en pixels
Left = pixels	Posición desde lado izquierdo, en pixels

Puedes utilizar el método open con múltiples atributos. Para ello, debes separarlos por comas, sin espacio.

La figura 17 muestra un código JavaScript que utiliza el método window.open para abrir una nueva ventana del navegador con una serie de atributos específicos.

Figura 17

Ejemplo de open con múltiples atributos

```
let url = "https://www.ejemplo.com";  
let nombreVentana = "MiVentana";  
let atributos = "width=400,height=300,toolbar=no,location=no,menubar=no,status=no,resizable=yes,scrollbars=yes";  
  
let nuevaVentana = window.open(url, nombreVentana, atributos);
```

¿Cómo implementar funcionalidades para abrir y cerrar ventanas?

A continuación, se presenta un ejemplo sobre cómo implementar funcionalidades para abrir y cerrar ventanas del navegador mediante botones en una página web, utilizando los métodos `open` y `close`.

En la figura 18 se muestran dos funciones:

- La primera función, `abreVentana`, permite abrir una nueva ventana. Dentro de ella se carga el sitio `www.duoc.cl`. El nombre que debe aparecer en el título de la ventana es “Nueva ventana”. Finalmente, debe mostrar el estado. Los atributos definidos permiten que la ventana sea redimensionada y establece que debe tener una ancho y alto de 100 pixeles.
- La segunda función llama al método `close`. Por tanto, se cierra la ventana abierta por la primera función.

Figura 18

Ejemplo uso de método `open` y método `close`

```
let v1;

function abreVentana() {
  v1 = window.open("http://www.duoc.cl", "Nueva Ventana", "status=yes,resizable=yes,width=100,height=100");
}

function cierraVentana() {
  v1.close();
}
```

Para ejecutar este código, agregamos lo siguiente en nuestra página principal:

Figura 19

Ejemplo de cómo gatillar las funciones

```
<input type="button" name="abrir" id="abrir" value="abrir" onclick="abreVentana()" />

<input type="button" name="cerrar" id="cerrar" value="cerrar" onclick="cierraVentana()" />
```



¡Haz la prueba!

Al presionar el botón “abrir”, se desplegará la pantalla en un recuadro muy pequeño debido a los atributos de tamaño especificados (100x100 pixeles).

Ahora, si presionas el botón “cerrar”, sin tocar la ventana recién abierta, la nueva ventana desaparecerá, ya que la función `cierraVentana()` se activará.

WINDOW - Ubicación

El objeto `location` contiene propiedades y métodos que permiten acceder y modificar la información relacionada con la dirección actual de la página. Su sintaxis es:

`var.location="url";`

Tabla 4

Atributos de location

Método	Descripción
assign(url)	Carga una nueva URL en la ventana actual.
Hash	Define o devuelve la parte del anclaje (todo lo que viene después del símbolo #) de una dirección URL
Host	Define o devuelve el nombre del host y el puerto
hostname	Define o devuelve el nombre del host de la URL (por ejemplo, "www.ejemplo.com")
Href	Define o devuelve la URL. Actualmente se recomienda usar location.toString() o location.assign()
Origin	Devuelve protocolo, hostname y el puerto de una URL
Pathname	Define o devuelve la ruta de la URL (por ejemplo, "/ruta/pagina.html")
Port	Define o devuelve el puerto de la URL
Protocol	Define o devuelve el protocolo URL (por ejemplo, "http:" o "https:")
Reload()	Recarga la página actual.
replace(url)	Reemplaza la URL actual con una nueva URL, lo que significa que no se agrega una entrada al historial de navegación.
Search	Define o devuelve el query de la URL (por ejemplo, "?parametro=valor")

A modo de ejemplo, veamos cómo se puede utilizar el objeto location para obtener información sobre la URL actual en una aplicación web y cómo se pueden desglosar sus componentes individuales.

En la figura 20 se muestra un fragmento de código JavaScript de una función llamada datosLocation(). Esta función utiliza propiedades del objeto location para obtener diferentes partes de la URL actual y las imprime en la consola.

Figura 20

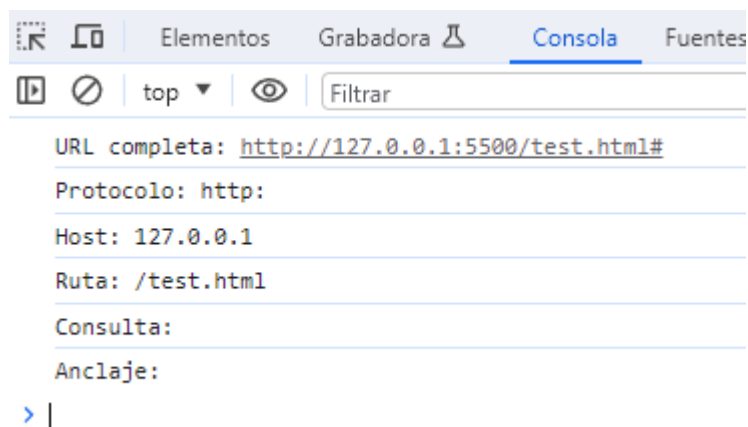
Ejemplo de location

```
function datoslocation() {  
    // Obtén y muestra información de la URL actual  
    let urlCompleta = location.href;  
    let protocolo = location.protocol;  
    let host = location.hostname;  
    let ruta = location.pathname;  
    let consulta = location.search;  
    let anclaje = location.hash;  
  
    console.log("URL completa: " + urlCompleta);  
    console.log("Protocolo: " + protocolo);  
    console.log("Host: " + host);  
    console.log("Ruta: " + ruta);  
    console.log("Consulta: " + consulta);  
    console.log("Anclaje: " + anclaje);  
}
```


En la figura 21 se muestra la consola de un navegador después de que se ejecutó la función `datosLocation()`.

Figura 21

Resultados al ejecutar la función `datosLocation()`



Recuerda: la información dependerá de la conexión local desde la que estás trabajando. Es decir, los datos que se obtienen a través del objeto `location`, como la URL completa, el host, la ruta, entre otros, varían según el entorno desde el cual el usuario está accediendo a la página web.

Window - History

El objeto **history** proporciona acceso y control sobre el historial de navegación del usuario en el navegador web. Esto permite interactuar con las páginas visitadas previamente en la

Tabla 5

Métodos del objeto history

Método	Descripción	Sintaxis
Back()	Vuelve a la página anterior	window.history.back()
Forward()	Va a la página siguiente	window.history.forward()
Go(valor)	Va a donde corresponda según valor: -1 = back N = lleva a la página n 1 = forward	window.history.go(valor)
pushState(state, title, url)	agrega una nueva entrada al historial de navegación con un nuevo estado, título y URL, sin forzar la carga de una nueva página.	window.history.pushState(stateObj, title, url);
replaceState(state, title, url)	Similar a pushState, este método reemplaza la entrada actual en el historial con una nueva entrada.	window.history.replaceState(stateData, "", newURL);

En el ejemplo de la figura 22, se utiliza el método back() para navegar una página hacia atrás en el historial de la sesión del navegador, mientras que el método forward se utiliza para navegar una página hacia adelante.

Figura 22

Ejemplo de uso de HISTORY

```
<button onclick="window.history.back()">atrás</button>  
<button onclick="window.history.forward()">adelante</button>
```

WINDOW – Manejo del tiempo

En JavaScript, puedes gestionar el tiempo de ejecución de tu código utilizando eventos y funciones específicas para controlar tareas que deben realizarse en el futuro.

Tabla 6

Métodos de manejo del tiempo

Método	Descripción	Sintaxis
setInterval()	Programa la ejecución repetitiva de una función cada cierto intervalo en milisegundos.	setInterval(función, intervalo)
setTimeout()	Programa la ejecución de una función después de un cierto retraso en milisegundos	setTimeout(función, retraso)
clearInterval()	Detiene la ejecución repetitiva de una función programada con setInterval	clearInterval(identificador)
clearTimeout()	Cancela la ejecución de una función programada con setTimeout antes de que ocurra	clearTimeout(identificador)

Para una mayor claridad sobre el uso de estos métodos, revisemos algunos ejemplos:

Ejemplo 1

En el ejemplo de la figura 23 se establece que la función se ejecute tras pasar 3 segundos. Sin embargo, antes de que pase el tiempo asignado y la función pueda ejecutarse, se invoca `clearTimeout`. Esto cancela la ejecución programada de `miFuncion`, asegurando que no se lleve a cabo.

Figura 23

Ejemplo de SETTIMEOUT y CLEARTIMEOUT

```
// Definir una función que se ejecutará después de un retraso
function miFuncion() {
    console.log("La función se ha ejecutado.");
}

// Programar la ejecución de la función después de 3 segundos
var timeoutID = setTimeout(miFuncion, 3000);

// Cancelar la ejecución de la función antes de que ocurra
clearTimeout(timeoutID);

console.log("La función ha sido cancelada.");
```

Entonces, en lugar de que en la consola se despliegue el texto “La función se ha ejecutado”, aparece “La función ha sido cancelada”:

Figura 24

Resultado de la función cancelada

La función ha sido cancelada.

Ejemplo 2

En el ejemplo de la figura 25, al utilizar setInterval la función se ejecuta cada dos segundos, y con el uso de clearInterval, se establece que la función se detendrá después de que han pasado 10 segundos.

Figura 25

Ejemplo de uso de SETINTERVAL

```
// Definir una función que se ejecutará repetidamente
function miFuncion() {
    console.log("La función se está ejecutando repetidamente.");
}

// Programar la ejecución de la función cada 2 segundos
var intervalID = setInterval(miFuncion, 2000);

// Detener la ejecución de la función después de 10 segundos
setTimeout(function() {
    clearInterval(intervalID);
    console.log("La función ha sido detenida.");
}, 10000);
```

Después de cancelar el intervalo con `clearInterval`, se imprime el mensaje "La función ha sido detenida."

Figura 26

Consola con resultado

```
5 La función se está ejecutando repetidamente.
  La función ha sido detenida.
>
```

Ejemplo 3

La figura 27 muestra un ejemplo de cómo imprimir un reloj que se actualiza en tiempo real usando `setInterval` para actualizar el reloj cada segundo, y así mantener la hora actualizada continuamente en la interfaz de usuario.

Figura 27

Código JavaScript que imprime un reloj que se va actualizando en tiempo real

```
var myVar = setInterval(function() {  
    reloj()  
}, 1000);  
  
function reloj() {  
    var d = new Date();  
  
    var t = d.toLocaleTimeString();  
    document.getElementById("hora").innerHTML = t;  
}
```

Para desplegar el reloj en una página o sitio web, se utiliza el código HTML que muestra la figura 28, el que define un encabezado <h1> con un atributo id establecido en "hora".

Figura 28

Código a agregar en página HTML para desplegar el reloj

```
<h1 id="hora"></h1>
```

El resultado se ve así:

15:05:13

SCREEN

El objeto **screen** proporciona información sobre la pantalla del usuario, como su resolución, profundidad de color y tamaño. Si bien puede obtener valores, NO puede modificarlos.

Tabla 7

Métodos del objeto screen

Método	Descripción	Sintaxis
Height	Altura de la pantalla.	var=screen.height
Width	Ancho de la pantalla.	var=screen.width
colorDepth	Profundidad de color (bits) en pantalla	var=screen.colorDepth
pixelDepth	Resolución de color (bits por píxel) de la pantalla	var=screen.pixelDepth
availHeight	Altura de la pantalla sin los taskbar	var=screen.availHeight
availWidth	Ancho de la pantalla sin los taskbar	var=screen.availWidth

En la figura 29 se muestra una función en JavaScript llamada datosScreen(). Esta función utiliza el objeto console.log para imprimir información sobre la pantalla del usuario, obtenida a través del objeto screen.

Figura 29

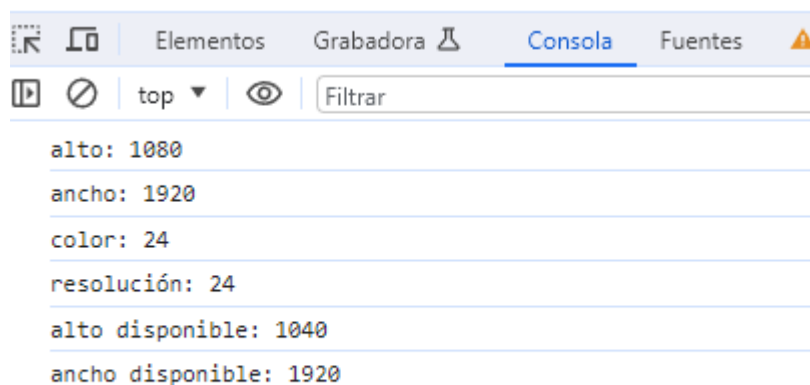
Ejemplo de SCREEN

```
function datosScreen() {  
    console.log("alto: " + screen.height);  
    console.log("ancho: " + screen.width);  
    console.log("color: " + screen.colorDepth);  
    console.log("resolución: " + screen.pixelDepth);  
    console.log("alto disponible: " + screen.availHeight);  
    console.log("ancho disponible: " + screen.availWidth);  
}
```

Al ejecutar esta función, los valores se mostrarán en la consola de desarrollo del navegador, tal como se muestra en la figura 30.

Figura 30

Despliegue de los resultados de la función en la consola



NAVIGATOR

El objeto **navigator** proporciona información sobre el navegador web y el entorno en el que se está ejecutando una aplicación web. A través de él puedes acceder a una variedad de propiedades y métodos que te permiten obtener información sobre el navegador, el dispositivo y las capacidades del navegador cliente.

Tabla 8

Propiedades del objeto Navigator

Método	Descripción	Sintaxis
cookieEnabled	Indica si las cookies están habilitadas en el navegador	var=navigator.cookieEnabled
appName	Devuelve el nombre del navegador	var=navigator.appName
appVersion	Devuelve la versión del navegador	var=navigator.appVersion
language	Devuelve el idioma preferido del usuario	var=navigator.language
platform	Devuelve la plataforma en la que se ejecuta el navegador	var=navigator.platform
userAgent	Devuelve el encabezado de agente de usuario enviado por el navegador al servidor	var=navigator.userAgent

Tabla 9

Métodos del objeto navigator

Método	Descripción	Sintaxis
getCurrentPosition()	Permite acceder a la ubicación geográfica del usuario, si este lo autoriza	navigator.geolocation.getCurrentPosition()
getUserMedia()	Permite acceder a dispositivos multimedia, como cámaras y micrófonos, para habilitar funciones como videoconferencias y captura de imágenes	navigator.mediaDevices.getUserMedia()
vibrate()	Activa la vibración del dispositivo (si está disponible) para proporcionar retroalimentación háptica	navigator.vibrate()
getHighEntropyValues())	Devuelve información de alta entropía que puede utilizarse para la detección de huellas digitales de dispositivos	navigator.userAgentData.getHighEntropyValues()

Revisemos algunos ejemplos de uso del objeto navegador:

Ejemplo de uso de la propiedad language del objeto navigator

En la figura 31 se declara una variable idioma y se le asigna el valor de navigator.language, que devuelve el idioma configurado en el navegador del usuario. Luego, se imprime este valor en la consola con console.log.

Figura 31

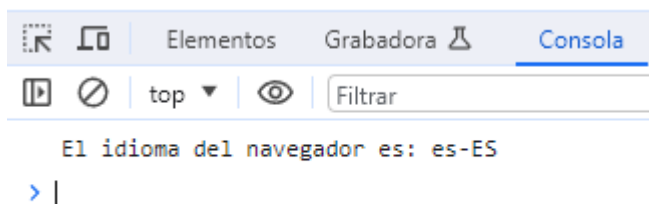
Ejemplo de NAVIGATOR

```
var idioma = navigator.language;  
console.log("El idioma del navegador es: " + idioma);
```

Al ejecutar el código en la consola del navegador, se visualiza el texto: "El idioma del navegador es: es-ES". Es decir, el idioma preferido del navegador está configurado en español de España (es-ES).

Figura 32

El resultado en consola es:



Ejemplo para verificar si las cookies están activas

La figura 33 muestra un bloque de código en JavaScript que utiliza la propiedad cookieEnabled del objeto navigator para verificar si las cookies están habilitadas en el navegador del usuario. Si navigator.cookieEnabled devuelve true, se imprime en la consola del navegador el mensaje "Las cookies están habilitadas en el navegador". Si devuelve false, se imprime el mensaje "Las cookies no están habilitadas en el navegador".

Figura 33

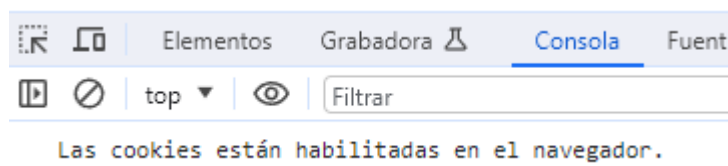
Ejemplo de decisión dependiendo de si las cookies están activadas

```
if (navigator.cookieEnabled) {  
    console.log("Las cookies están habilitadas en el navegador.");  
} else {  
    console.log("Las cookies no están habilitadas en el navegador.");  
}
```

La figura 34 muestra el resultado al ejecutar el código en un navegador donde las cookies están habilitadas. En este caso, el mensaje impreso en la consola es: "Las cookies están habilitadas en el navegador".

Figura 34

Mensaje impreso en la consola



Ejemplo que permite obtener la geolocalización del usuario

El código de la figura 35 señala que se preguntará si el usuario permite que el navegador obtenga esa información, y posteriormente la imprime en la consola:

Figura 35

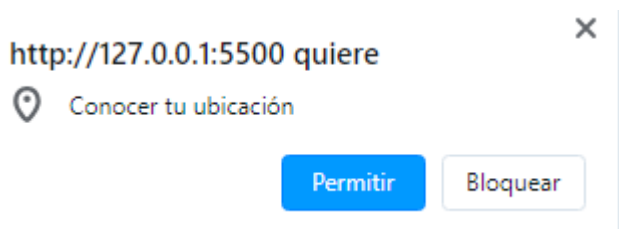
Ejemplo de GEOLOCATION

```
if ("geolocation" in navigator) {  
    navigator.geolocation.getCurrentPosition(function(position) {  
        let latitud = position.coords.latitude;  
        let longitud = position.coords.longitude;  
        console.log("Tu ubicación actual es: Latitud " + latitud + ", Longitud " + longitud);  
    }, function(error) {  
        console.error("Error al obtener la ubicación: " + error.message);  
    });  
} else {  
    console.log("Geolocalización no está disponible en este navegador.");  
}
```

Al cargar la página, el navegador ejecuta su política de seguridad automática, preguntando si deseamos dar la información a la página web y tal como se muestra en la figura 36, el usuario tiene la opción de "Permitir" o "Bloquear" el acceso a su ubicación por parte de la página web.

Figura 36

Despliegue de consulta de seguridad por parte del navegador



Al presionar aceptar, se obtiene la información y se despliega en la consola un mensaje que indica la latitud y la longitud obtenida.

Figura 37

Mensaje en la consola

```
Tu ubicación actual es: Latitud -33.4714352, Longitud -70.5927073  
>
```

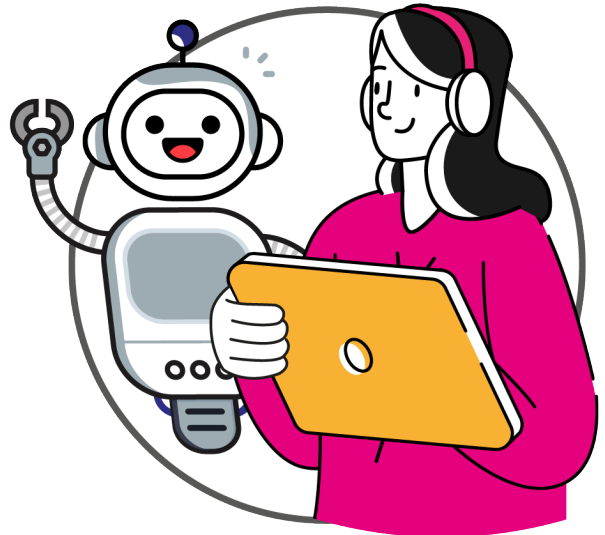
Cierre de la semana

Esta semana has aprendido a obtener información del cliente utilizando distintos métodos y atributos de JavaScript. Aprendiste que el BOM significa Browser Object Model (o Modelo de Objetos del Navegador), que permiten manipular ventanas, urls, historial de navegación y los tiempos en que se ejecutan los programas de JavaScript.

Además, lograste imprimir desde el navegador, usando `print()` y CSS para obtener páginas que tienen un diseño distinto en el navegador y en el papel.

Con estos conocimientos estás listo para aprender la parte más entretenida de JavaScript: la modificación de los elementos de una página web.

¡Nos vemos en la próxima experiencia!



Referencias

- Ramírez, L. (s.f.). *JavaScript, 5 cosas que hay que saber*. <http://js.luisfel.cl>
- Mozilla Web Docs. (s.f.). *Guía de JavaScript*.
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Introducci%C3%B3n>

This image shows a full page of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page, providing a template for writing. There are no margins, text, or other markings on the page.



DuocUC[®] ONLINE



Facilitador disciplinar: Claudio Navarro

Asesora par: Paola Véliz

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.